

Université Abdelmalek Essaâdi
École Nationale des Sciences Appliquées – Al Hoceima

Compte Rendu

TP 5 - Java

Programmation Orientée Objet ID1 – S2 / 2025-26

Réalisé par : [Soukaina Ouyadline](#)

Présenté à : [Pr. Mohamed CHERRADI](#)

PARTIE 1 — Exceptions système (surveillées & non surveillées) :

Exercice 1 (Division sécurisée) :

```
package exercice1.java;
public class Calcul {
    public static int Diviser(int a,int b) {
        if (b == 0) {
            throw new ArithmeticException("Division par zéro impossible");
        }
        return a / b;
    }
}
```

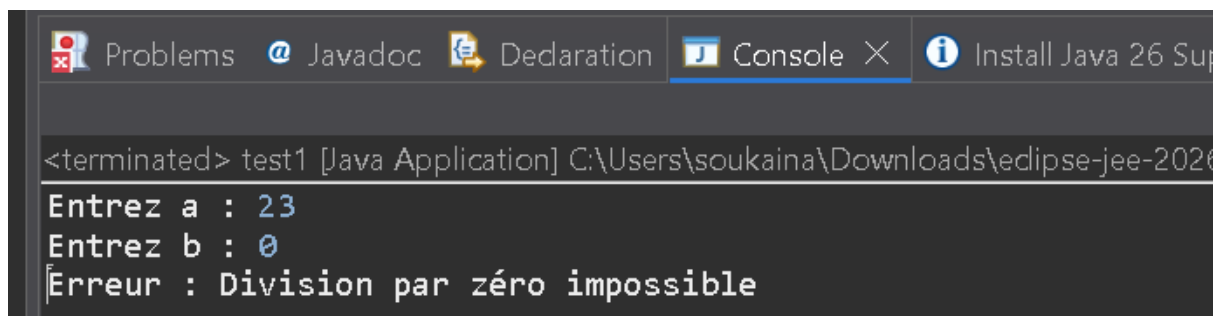
```
package exercice1.java;
import java.util.Scanner;

public class test1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Entrez a : ");
            int a = sc.nextInt();

            System.out.print("Entrez b : ");
            int b = sc.nextInt();

            int resultat = Calcul.Diviser(a, b);
            System.out.println("Résultat = " + resultat);
        } catch (ArithmeticException e) {
            System.out.println("Erreur : " + e.getMessage());
        } catch (InputMismatchException e) {
            System.out.println("Erreur : saisie nombre invalide ");
        }
    }
}
```



```
<terminated> test1 [Java Application] C:\Users\soukaina\Downloads\edipse-jee-2026
Entrez a : 23
Entrez b : 0
Erreur : Division par zéro impossible
```

Exercice 2 (NullPointerException) :

```
1 package exercice2.java;
2
3 public class Longueur1 {
4
5     // sans try/catch
6     public static void afficherLongueur(String texte) {
7
8         if (texte == null) {
9             System.out.println("La chaîne est null");
10        } else {
11            System.out.println("Longueur : " + texte.length());
12        }
13    }
14 }
```

```
package exercice2.java;
```

```

public class Longueur2 {

    public static void afficherLongueurAvecTry(String texte) {
        try {
            System.out.println("Longueur : " + texte.length());
        } catch (NullPointerException e) {
            System.out.println("Erreur : la chaîne est null");
        }
    }
}

```

```

package exercice2.java;
public class main {
    public static void main(String[] args) {

        Longueur1.afficherLongueur("salut");
        Longueur1.afficherLongueur(null);

        Longueur2.afficherLongueurAvecTry("soukaina");
        Longueur2.afficherLongueurAvecTry(null);
    }
}

```

```

C:\Users\soukaina\Downloads> java -cp .\exercice2\exercice2.jar main
Longueur : 5
La chaîne est null
Longueur : 8
Erreur : la chaîne est null

```

Exercice 3 (Tableau sécurisé) :

```

public class Tableau_if {
    public static void accederElement(int[] tableau, int index) {
        if (index >= 0 && index < tableau.length) {
            System.out.println("Valeur : " + tableau[index]);
        } else {
            System.out.println("Erreur : index hors limites");
        }
    }
}

package exercice3;
public class Tableau_try {
    public static void accederElement(int[] tableau, int index) {
        try {
            System.out.println("Valeur : " + tableau[index]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Erreur : index hors limites");
        }
    }
}
package exercice3;

```

```

public class main {
    public static void main(String[] args) {

        int[] tableau = {10, 20, 30, 40, 50};
        // if
        Tableau_if.accederElement(tableau, 5);
        Tableau_if.accederElement(tableau, 3);
        // try/catch
        Tableau_try.accederElement(tableau, 1);
        Tableau_try.accederElement(tableau, 4);
    }
}

```

```

<terminated> main (10) [Java Application] C:\Users\soukaina\Download
Erreur : index hors limites
Valeur : 40
Valeur : 20
Valeur : 50

```

Exercice 4 (Conversion d'entier) :

```

package exercice4;
public class conversion_try {
    public static void convertir(String texte) {
        try {
            int nombre = Integer.parseInt(texte);
            System.out.println("Nombre converti : " + nombre);
        } catch (NumberFormatException e) {
            System.out.println("Erreur : impossible de convertir en entier");
        }
    }
}

```

```

package exercice4;
public class conversion {
    public static void convertir(String texte) {

        if (texte.matches("-?\\d+")) {
            int nombre = Integer.parseInt(texte);
            System.out.println("Nombre converti : " + nombre);
        } else {
            System.out.println("Erreur : la chaîne n'est pas un entier valide");
        }
    }
}
package exercice4;

```

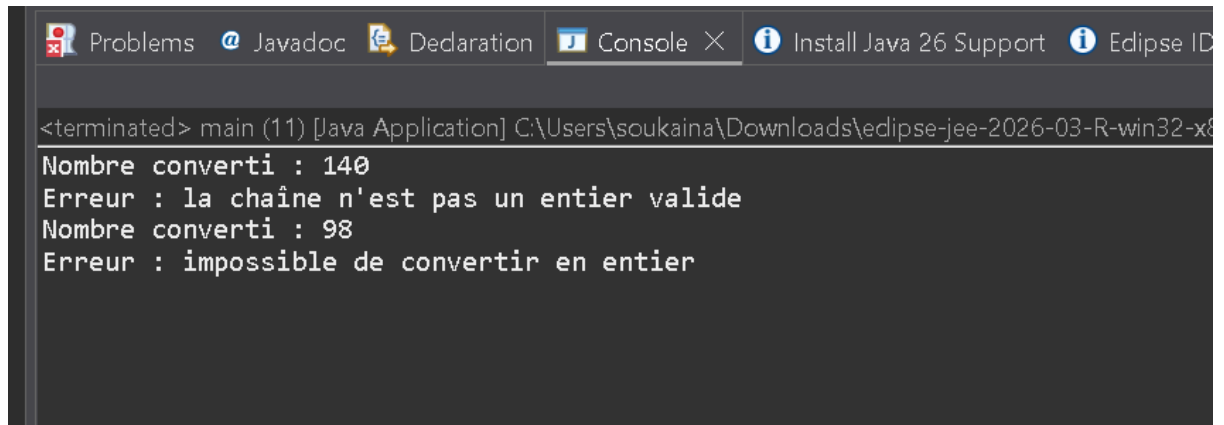
```

public class main {
    public static void main(String[] args) {

        conversion.convertir("140");
        conversion.convertir("sakina");

        conversion_try.convertir("098");
        conversion_try.convertir("ouyadline");
    }
}

```



```

<terminated> main (11) [Java Application] C:\Users\soukaina\Downloads\edipse-jee-2026-03-R-win32-x86_64.exe;
Nombre converti : 140
Erreur : la chaîne n'est pas un entier valide
Nombre converti : 98
Erreur : impossible de convertir en entier

```

Exercice 5 (Choix d'exception) :

```

package exercice5;
public class math {
    public static int racineCarree(int x) {

        if (x < 0) {
            throw new IllegalArgumentException(
                "Erreur : x doit être positif ou nul"
            );
        }

        return (int) Math.sqrt(x);
    }
}

```

```

package exercice5;
public class main {
    public static void main(String[] args) {
        System.out.println(
            math.racineCarree(36)
        );
        System.out.println(
            math.racineCarree(-1)
        );
    }
}

```

```
10
11     System.out.println(
12         math.racineCarree(-1)
13     );
14 }
15
```

Problems Javadoc Declaration Console X Install Java 26 Support Eclipse IDE for Enterprise Java and Web T

<terminated> main (12) [Java Application] C:\Users\soukaina\Downloads\edipse-jee-2026-03-R-win32-x86_64\edipse\plugins\org.edips

6

Exception in thread "main" java.lang.IllegalArgumentException: Erreur : x doit être positif ou nul
at exercice5.math.racineCarree(math.java:10)
at exercice5.main.main(main.java:12)

Exercice 6 (IllegalStateException) :

```
package exercice6;
public class Machine {

    private boolean allumee = false;
    public void demarrer() {

        if (allumee) {
            throw new IllegalStateException(
                "Erreur : la machine est déjà démarrée"
            );
        }

        allumee = true;
        System.out.println(" Machine démarrée");
    }
}
```

```
package exercice6;
public class main {
    public static void main(String[] args) {
        Machine machine = new Machine();
        try {
            machine.demarrer();
            machine.demarrer();
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
15  
16 System.out.println(e.getMessage());  
17  
Problems Javadoc Declaration Console X Install Java 26 Support Eclipse IDE for Enterprise Java and Web De  
<terminated> main (13) [Java Application] C:\Users\soukaina\Downloads\eclipse-jee-2026-03-R-win32-x86_64\eclipse\plugins\org.eclipse  
Machine démarrée  
Erreur : la machine est déjà démarrée
```

Exercice 7 (Propagation) :

```
package exercice7;  
public class Propagation {  
  
    public static void methodeA() {  
        try {  
            methodeB();  
        } catch (ArithmeticException e) {  
            System.out.println(  
                "Exception capturée dans A : " + e.getMessage()  
            );  
        }  
    }  
  
    public static void methodeB() {  
        methodeC();  
    }  
  
    public static void methodeC() {  
        throw new ArithmeticException(  
            "Division par zéro"  
        );  
    }  
}
```

```
package exercice7;  
public class main {  
    public static void main(String[] args) {  
        Propagation.methodeA();  
    }  
}
```

```
Problems Javadoc Declaration Console X Install Java 26 S
<terminated> main (14) [Java Application] C:\Users\soukaina\Downloads\edipse-je
Exception capturée dans A : Division par zéro
```

Exercice 8 (Checked Exception) :

```
package exercice8;
public class Age {

    public static void verifierAge(int age) throws Exception {

        if (age < 18) {
            throw new Exception(
                "Erreur : âge inférieur à 18"
            );
        }
        System.out.println("Accès autorisé");
    }
}
```

```
package exercice8;
public class main {
    public static void main(String[] args) {
        try {
            Age.verifierAge(22);
            Age.verifierAge(4);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```
5
6
7
8
ava
java
Library [re]
Problems Javadoc Declaration Console X Install Java 26 Sup
<terminated> main (15) [Java Application] C:\Users\soukaina\Downloads\edipse-jee-2
Accès autorisé
Erreur : âge inférieur à 18
```


PARTIE 2 — Exceptions personnalisées (checked & unchecked)

Exercice 1 (Compte bancaire) :

```
package exercice1;
public class CompteBancaire {
    private String code;
    private double solde;
    public CompteBancaire(String code, double solde) {
        this.code = code;
        this.solde = solde;
    }

    public void verser(double montant) {
        if (montant > 0) {
            solde += montant;
            System.out.println("Versement effectué : " + montant);
        }
    }

    public void retirer(double montant) throws SoldeInsuffisantException {
        if (montant > solde) {
            throw new SoldeInsuffisantException(
                "Solde insuffisant : opération refusée"
            );
        }
        solde -= montant;
        System.out.println("Retrait effectué : " + montant);
    }
    public double getSolde() {
        return solde;
    }
}
```

```
package exercice1;
public class SoldeInsuffisantException extends Exception {
    public SoldeInsuffisantException(String message) {
        super(message);
    }
}
```

```
package exercice1;
public class main {
    public static void main(String[] args) {
        CompteBancaire compte = new CompteBancaire("C001", 100);
        compte.verser(200);
        try {
            compte.retirer(245);
            compte.retirer(88);
        }
    }
}
```

```

    } catch (SoldeInsuffisantException e) {
        System.out.println(e.getMessage());
    }
    System.out.println("Solde final : " + compte.getSolde());
}
}

```

```

<terminated> main (16) [Java Application] C:\Users\soukaina\Downloads\edipse-j;
Versement effectué : 200.0
Retrait effectué : 245.0
Solde insuffisant : opération refusée
Solde final : 55.0

```

Exercice 2 (Montant invalide) :

```

package exercice2;
public class CompteBancaire {
    private String code;
    private double solde;
    public CompteBancaire(String code, double solde) {
        this.code = code;
        this.solde = solde;
    }
    public void verser(double montant) throws MontantInvalideException {
        if (montant <= 0) {
            throw new MontantInvalideException(
                "Montant invalide : doit être > 0"
            );
        }
        solde += montant;
        System.out.println("Versement : " + montant);
    }

    public void retirer(double montant)
        throws MontantInvalideException {
        if (montant <= 0) {
            throw new MontantInvalideException(
                "Montant invalide : doit être > 0"
            );
        }
        if (montant > solde) {
            System.out.println("Solde insuffisant");
            return;
        }
    }
}

```

```

    }
    solde -= montant;
    System.out.println("Retrait : " + montant);
}
public double getSolde() {
    return solde;
}
}

```

```

package exercice2;
public class MontantInvalideException extends Exception {
    public MontantInvalideException(String message) {
        super(message);
    }
}

```

```

package exercice2;
public class main {
    public static void main(String[] args) {
        CompteBancaire compte = new CompteBancaire("C002", 800);
        try {
            compte.verser(100);
            compte.verser(-30);
        } catch (MontantInvalideException e) {
            System.out.println(e.getMessage());
        }
        try {
            compte.retirer(70);
            compte.retirer(0);
        } catch (MontantInvalideException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Solde final : " + compte.getSolde());
    }
}

```

```
Exc... Problems Javadoc Declaration Con...
<terminated> main (17) [Java Application] C:\Users\sc...
Versement : 100.0
Montant invalide : doit être > 0
Retrait : 70.0
Montant invalide : doit être > 0
Solde final : 830.0
```

Exercice 3 (Double validation) :

```
package exercice3;
public class CompteBancaire {
    private String code;
    private double solde;

    public CompteBancaire(String code, double solde) {
        this.code = code;
        this.solde = solde;
    }
    public String getCode() {
        return code;
    }
    public double getSolde() {
        return solde;
    }

    public void verser(double montant) throws MontantInvalideException {
        if (montant <= 0) {
            throw new MontantInvalideException("Montant invalide : doit être > 0");
        }
        solde += montant;
    }

    public void retirer(double montant)
        throws MontantInvalideException, SoldeInsuffisantException {
        if (montant <= 0) {
            throw new MontantInvalideException("Montant invalide : doit être > 0");
        }
        if (montant > solde) {
            throw new SoldeInsuffisantException("Solde insuffisant");
        }
        solde -= montant;
    }
}
```

```

    }
    public void afficher() {
        System.out.println("Compte: " + code + " | Solde: " + solde);
    }
}

```

```

package exercice3;
public class MontantInvalideException extends Exception {
    public MontantInvalideException(String message) {
        super(message);
    }
}

```

```

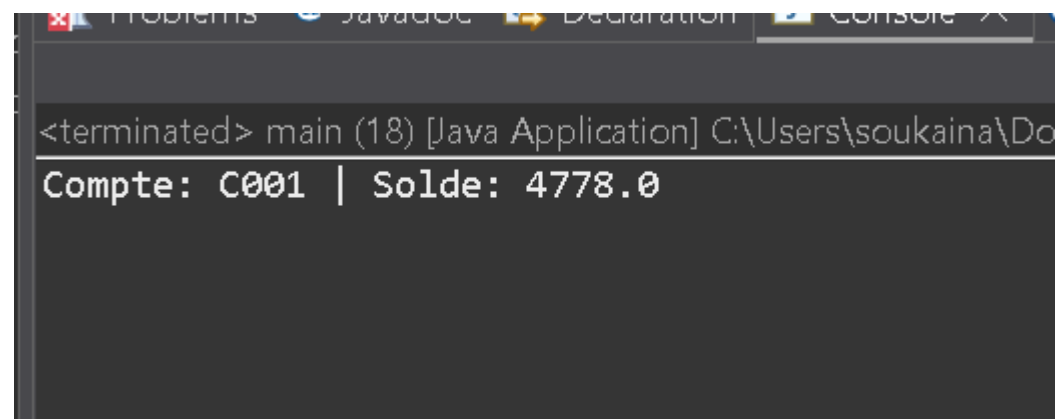
package exercice3;
public class SoldeInsuffisantException extends Exception {
    public SoldeInsuffisantException(String message) {
        super(message);
    }
}

```

```

package exercice3;
public class main {
    public static void main(String[] args) {
        CompteBancaire compte = new CompteBancaire("C001", 9000);
        try {
            compte.verser(123);
            compte.retirer(345);
            compte.retirer(4000);
        } catch (MontantInvalideException | SoldeInsuffisantException e) {
            System.out.println("Erreur: " + e.getMessage());
        }
        compte.afficher();
    }
}

```



The screenshot shows an IDE window with a tab labeled 'Console'. The console output displays the result of the program's execution, showing the account details after a series of transactions.

```

<terminated> main (18) [Java Application] C:\Users\soukaina\Do
Compte: C001 | Solde: 4778.0

```

Exercice 4 (Inscription Utilisateur) :

```
package exercice4;
public class EmailInvalideException extends Exception {
    public EmailInvalideException(String message) {
        super(message);
    }
}
```

```
package exercice4;
public class Exception_age extends Exception {

    public Exception_age(String message) {
        super(message);
    }
}
```

```
package exercice4;
public class utilisateur {
    private String email;
    private int age;

    public void inscrire(String email, int age)
        throws EmailInvalideException, Exception_age {

        if (email == null || !email.contains("@") || !email.contains(".")) {
            throw new EmailInvalideException("Email invalide");
        }

        if (age < 18) {
            throw new Exception_age("Âge invalide : doit être >= 18 ans");
        }

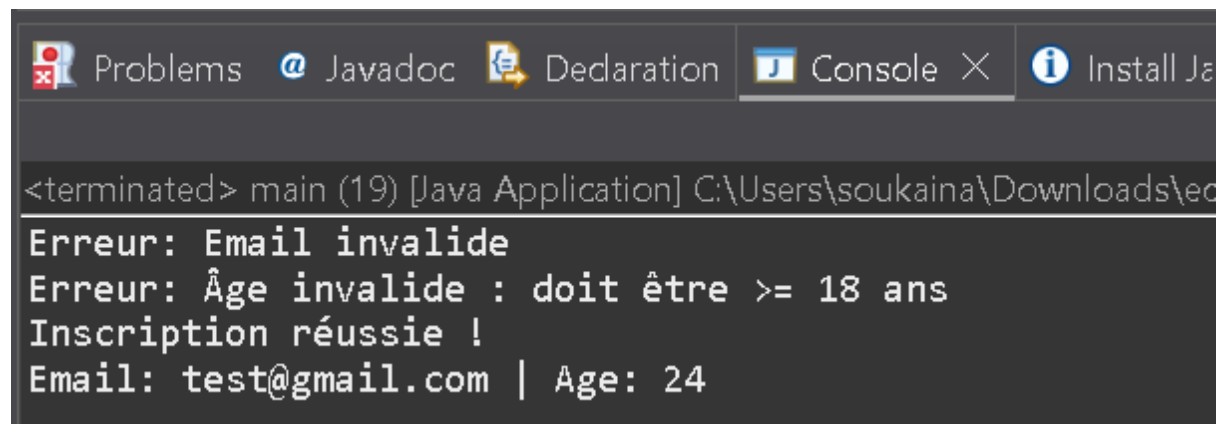
        this.email = email;
        this.age = age;
        System.out.println("Inscription réussie !");
    }
    public void afficher() {
        System.out.println("Email: " + email + " | Age: " + age);
    }
}
```

```
package exercice4;
public class main {
    public static void main(String[] args) {
        utilisateur u = new utilisateur();
        try {
            u.inscrire("testgmail.com", 36);
        }
    }
}
```

```

    } catch (EmailInvalideException | Exception_age e) {
        System.out.println("Erreur: " + e.getMessage());
    }
    try {
        u.inscrire("test@gmail.com", 16);
    } catch (EmailInvalideException | Exception_age e) {
        System.out.println("Erreur: " + e.getMessage());
    }
    try {
        u.inscrire("test@gmail.com", 24);
    } catch (EmailInvalideException | Exception_age e) {
        System.out.println("Erreur: " + e.getMessage());
    }
    u.afficher();
}
}

```



```

<terminated> main (19) [Java Application] C:\Users\soukaina\Downloads\ec
Erreur: Email invalide
Erreur: Âge invalide : doit être >= 18 ans
Inscription réussie !
Email: test@gmail.com | Age: 24

```

Exercice 5 (Authentification) :

```

package exercice5;
public class AuthentificationException extends Exception {
    public AuthentificationException(String message) {
        super(message);
    }
}

```

```

package exercice5;
public class AuthService {

    public void login(String username, String password)
        throws AuthentificationException {

        String correctUsername = "admin";
        String correctPassword = "1234";
    }
}

```

```

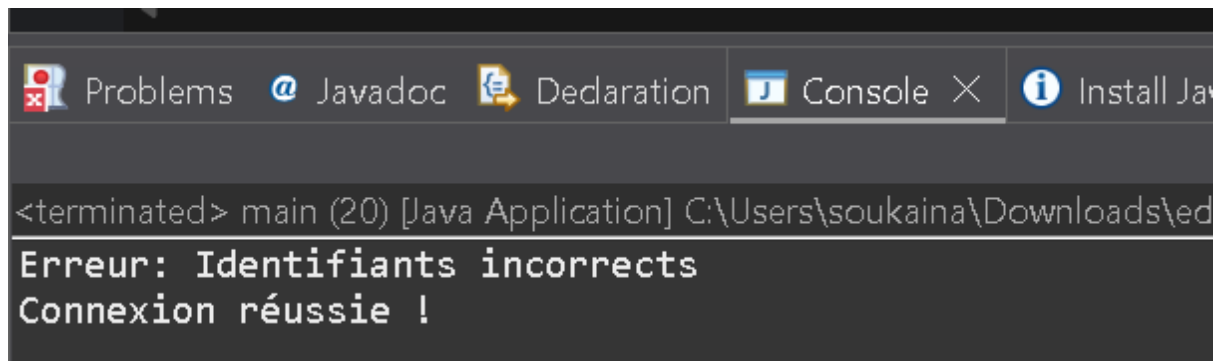
        if (!username.equals(correctUsername) || !password.equals(correctPassword)) {
            throw new AuthenticationException("Identifiants incorrects");
        }
        System.out.println("Connexion réussie !");
    }
}

```

```

package exercice5;
public class main {
    public static void main(String[] args) {
        AuthService auth = new AuthService();
        try {
            auth.login("user", "0000");
        } catch (AuthenticationException e) {
            System.out.println("Erreur: " + e.getMessage());
        }
        try {
            auth.login("admin", "1234");
        } catch (AuthenticationException e) {
            System.out.println("Erreur: " + e.getMessage());
        }
    }
}

```



The screenshot shows the IDE's console window with the following output:

```

<terminated> main (20) [Java Application] C:\Users\soukaina\Downloads\ed
Erreur: Identifiants incorrects
Connexion réussie !

```

Exercice 6 (Gestion de stock) :

```

package exercice6;
public class Produit {
    private String nom;
    private int stock;
    public Produit(String nom, int stock) {
        this.nom = nom;
        this.stock = stock;
    }

    public void retirerDuStock(int quantite) throws StockInsuffisantException {

```



```

        if (quantite > stock) {
            throw new StockInsuffisantException("Stock insuffisant");
        }

        stock -= quantite;
    }

    public void afficher() {
        System.out.println("Produit: " + nom + " | Stock: " + stock);
    }
}

```

```

package exercice6;

public class StockInsuffisantException extends Exception {
    public StockInsuffisantException(String message) {
        super(message);
    }
}

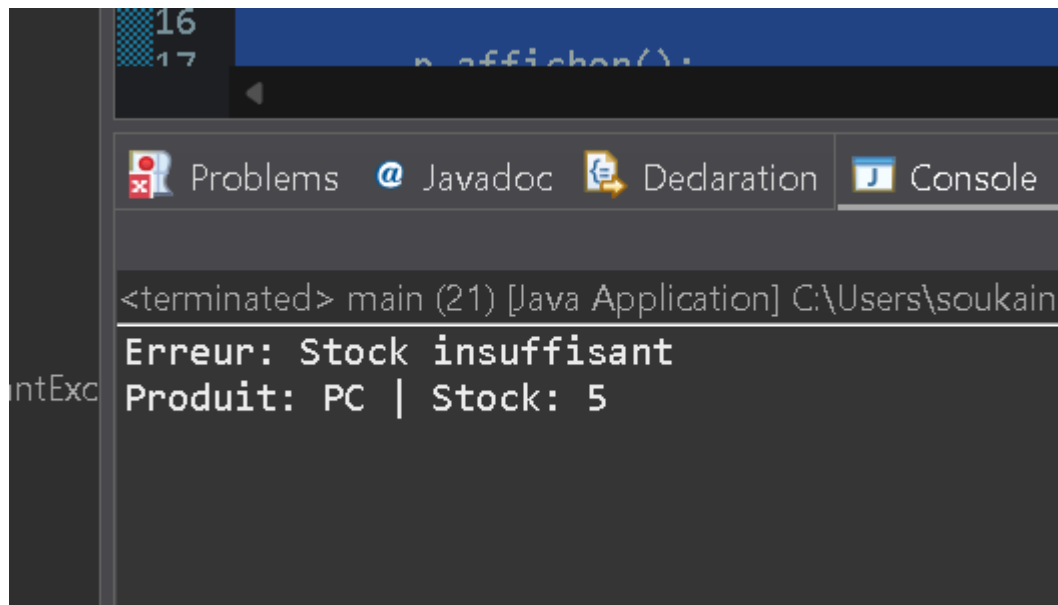
```

```

package exercice6;

public class main {
    public static void main(String[] args) {
        Produit p = new Produit("PC", 10);
        try {
            p.retirerDuStock(5);
            p.retirerDuStock(20);
        } catch (StockInsuffisantException e) {
            System.out.println("Erreur: " + e.getMessage());
        }
        p.afficher();
    }
}

```



Exercice 7 (Téléchargement) :

```
package exercice7;
public class GestionTelechargement {
    private double limite = 100.0;

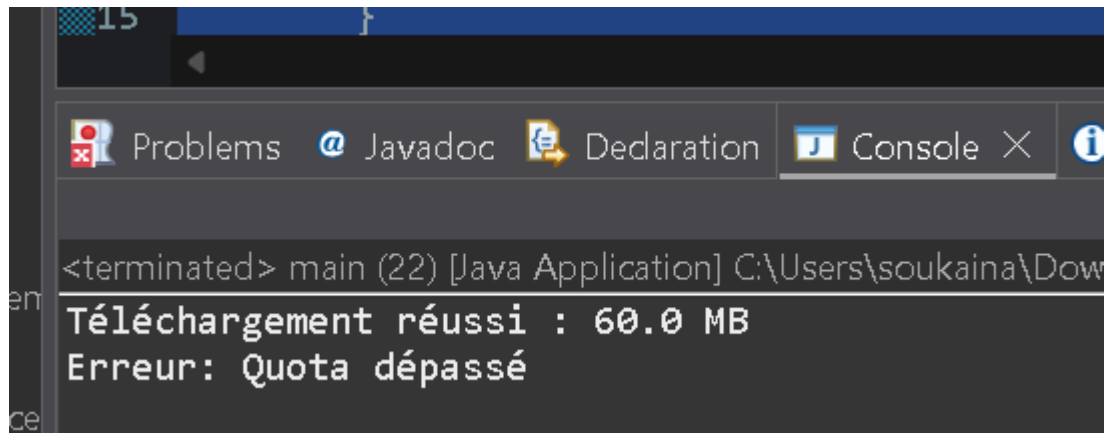
    public void telechargerFichier(double taille) throws QuotaDepasseException {

        if (taille > limite) {
            throw new QuotaDepasseException("Quota dépassé");
        }
        System.out.println("Téléchargement réussi : " + taille + " MB");
    }
}
```

```
package exercice7;
public class QuotaDepasseException extends Exception {
    public QuotaDepasseException(String message) {
        super(message);
    }
}

package exercice7;
public class main {
    public static void main(String[] args) {
        GestionTelechargement g = new GestionTelechargement();
        try {
            g.telechargerFichier(60);
            g.telechargerFichier(430);
        } catch (QuotaDepasseException e) {
            System.out.println("Erreur: " + e.getMessage());
        }
    }
}
```

```
}
```



Exercice 8 (Validation de formulaire) :

```
package exercice8;
public class ChampObligatoireException extends Exception {
    public ChampObligatoireException(String message) {
        super(message);
    }
}
```

```
package exercice8;
public class Formulaire {

    public void validerFormulaire(String nom, String email)
        throws ChampObligatoireException {

        if (nom == null || nom.trim().isEmpty()) {
            throw new ChampObligatoireException("Le champ nom est obligatoire");
        }

        if (email == null || email.trim().isEmpty()) {
            throw new ChampObligatoireException("Le champ email est obligatoire");
        }
        System.out.println("Formulaire valide !");
    }
}
```

```
package exercice8;
public class main {
    public static void main(String[] args) {
        Formulaire f = new Formulaire();
        try {
            f.validerFormulaire("", "test@gmail.com");
        } catch (ChampObligatoireException e) {
            System.out.println("Erreur: " + e.getMessage());
        }
    }
}
```

```

try {
    f.validerFormulaire("sakina", "");
} catch (ChampObligatoireException e) {
    System.out.println("Erreur: " + e.getMessage());
}
try {
    f.validerFormulaire("sakina", "test@gmail.com");
} catch (ChampObligatoireException e) {
    System.out.println("Erreur: " + e.getMessage());
}
}
}

```

```

13         System.out.println("Erreur: " + e.ge
14     }
15

```

Problems Javadoc Declaration Console X Install

<terminated> main (36) [Java Application] C:\Users\soukaina\Downloads\

Erreur: Le champ nom est obligatoire
 Erreur: Le champ email est obligatoire
 Formulaire valide !

Exercice 9 (Paieement) :

```

package exercice9;
public class CarteExpireeException extends Exception {
    public CarteExpireeException(String message) {
        super(message);
    }
}

```

```

package exercice9;
public class Paiement {
    private double plafond = 1000.0;
    private boolean carteExpiree;

    public Paiement(boolean carteExpiree) {
        this.carteExpiree = carteExpiree;
    }

    public void payer(double montant)

```

```

        throws PaiementRefuseException, CarteExpireeException {

        if (carteExpiree) {
            throw new CarteExpireeException("Carte expirée");
        }

        if (montant > plafond) {
            throw new PaiementRefuseException("Paiement refusé : plafond dépassé");
        }
        System.out.println("Paiement effectué : " + montant + " DH");
    }
}

```

```

package exercice9;
public class PaiementRefuseException extends Exception {
    public PaiementRefuseException(String message) {
        super(message);
    }
}

```

```

package exercice9;
public class main {
    public static void main(String[] args) {

        Paiement p1 = new Paiement(true);
        try {
            p1.payer(200);
        } catch (PaiementRefuseException | CarteExpireeException e) {
            System.out.println("Erreur: " + e.getMessage());
        }

        Paiement p2 = new Paiement(false);
        try {
            p2.payer(2000);
        } catch (PaiementRefuseException | CarteExpireeException e) {
            System.out.println("Erreur: " + e.getMessage());
        }

        try {
            p2.payer(234);
        } catch (PaiementRefuseException | CarteExpireeException e) {
            System.out.println("Erreur: " + e.getMessage());
        }
    }
}

```

```
Problems @ Javadoc Declaration Console X In
<terminated> main (23) [Java Application] C:\Users\soukaina\Downlo
Erreur: Carte expirée
Erreur: Paiement refusé : plafond dépassé
Paiement effectué : 234.0 DH
```

Exercice 10 (Conception) :

On utilise une **checked exception** lorsque l'erreur est prévisible et que le programme doit obligatoirement la gérer, par exemple avec un **try/catch**, comme les problèmes d'accès à un fichier ou une base de données. Les **unchecked exceptions**, elles, sont utilisées pour des erreurs de programmation ou de logique, comme une division par zéro ou un accès à une valeur **null**, et elles ne sont pas obligatoires à gérer. Enfin, on crée une **exception personnalisée** pour rendre le code plus clair et plus compréhensible, mieux représenter les erreurs du métier (comme âge invalide ou stock insuffisant), et faciliter la maintenance et la gestion des erreurs dans l'application.

PARTIE 3 — Généricité

Exercice 1 :

```
package exercice1;
public class Boite<T> {
    private T contenu;

    public void setContenu(T contenu) {
        this.contenu = contenu;
    }

    public T getContenu() {
        return contenu;
    }
}
```

```
package exercice1;
```

```

public class main {
    public static void main(String[] args) {

        Boite<String> boiteString = new Boite<>();
        boiteString.setContenu("salut");
        System.out.println("Boite String : " + boiteString.getContenu());

        Boite<Integer> boiteInteger = new Boite<>();
        boiteInteger.setContenu(44);
        System.out.println("Boite Integer : " + boiteInteger.getContenu());
    }
}

```

The screenshot shows an IDE with a code editor and a console window. The code editor displays the following lines of Java code:

```

12
13         Boite<Integer> boiteInteger = new Boite<>();
14         boiteInteger.setContenu(44);
15         System.out.println("Boite Integer : " + boiteInteger.getContenu());

```

Below the code editor, there is a toolbar with icons for Problems, Javadoc, Declaration, and Console. The Console window shows the output of the program:

```

terminated> main (24) [Java Application] C:\Users\s...
Boite String : salut
Boite Integer : 44

```

Exercice 2 :

```

package exercice2;

public class Paire<T, U> {
    private T premier;
    private U second;

    public Paire(T premier, U second) {
        this.premier = premier;
        this.second = second;
    }

    public void afficherPaire() {
        System.out.println("Premier : " + premier + " | Second : " + second);
    }
}

```

```

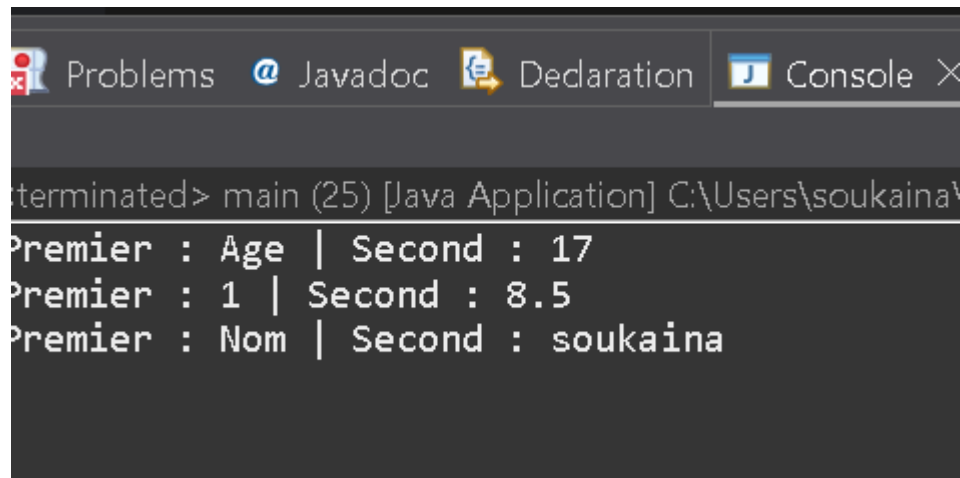
package exercice2;
public class main {
    public static void main(String[] args) {

        Paire<String, Integer> p1 = new Paire<>("Age", 17);
        p1.afficherPaire();

        Paire<Integer, Double> p2 = new Paire<>(1, 8.5);
        p2.afficherPaire();

        Paire<String, String> p3 = new Paire<>("Nom", "soukaina");
        p3.afficherPaire();
    }
}

```



```

terminated> main (25) [Java Application] C:\Users\soukaina\
Premier : Age | Second : 17
Premier : 1 | Second : 8.5
Premier : Nom | Second : soukaina

```

Exercise 3 :

```

package exercice3;
public interface Calcul<T> {
    T addition(T a, T b);
}

```

```

package exercice3;
public class CalculDouble implements Calcul<Double> {
    @Override
    public Double addition(Double a, Double b) {
        return a + b;
    }
}

```

```

package exercice3;
public class CalculInteger implements Calcul<Integer> {
    @Override

```



```

    public Integer addition(Integer a, Integer b) {
        return a + b;
    }
}

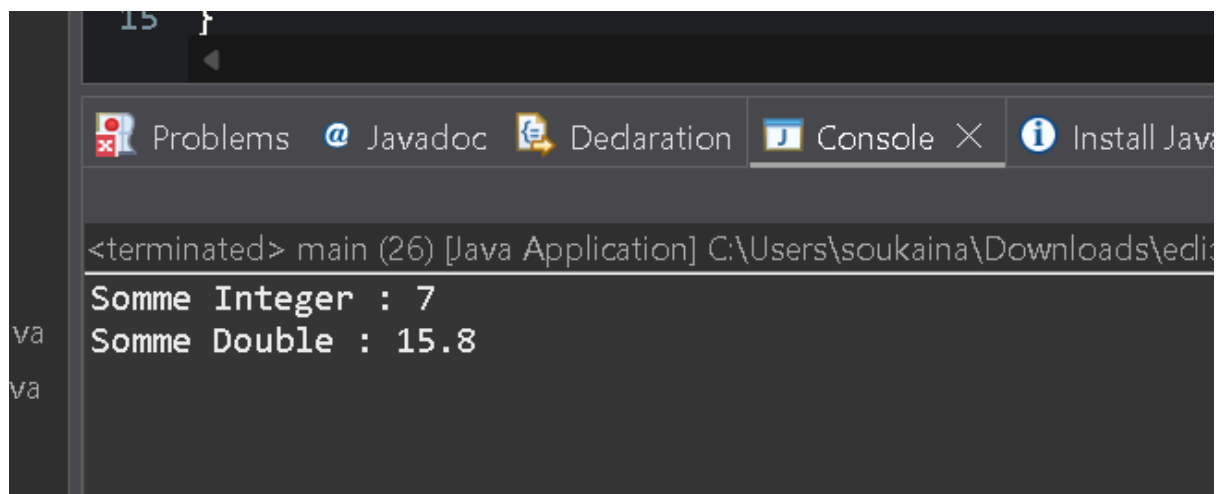
```

```

package exercice3;
public class main {
    public static void main(String[] args) {

        Calcul<Integer> calcInt = new CalculInteger();
        System.out.println("Somme Integer : " + calcInt.addition(5, 2));
        // Test Double
        Calcul<Double> calcDouble = new CalculDouble();
        System.out.println("Somme Double : " + calcDouble.addition(7.5, 8.3));
    }
}

```



```

<terminated> main (26) [Java Application] C:\Users\soukaina\Downloads\edit
Somme Integer : 7
Somme Double : 15.8

```

Exercice 4 :

```

package exercice4;
public interface Comparateur<T> {
    int comparer(T a, T b);
}

```

```

package exercice4;
public class ComparateurInteger implements Comparateur<Integer> {
    @Override
    public int comparer(Integer a, Integer b) {
        return a.compareTo(b);
    }
}

```

```

package exercice4;
public class CompareurString implements Compareur<String> {
    @Override
    public int comparer(String a, String b) {
        return Integer.compare(a.length(), b.length());
    }
}

```

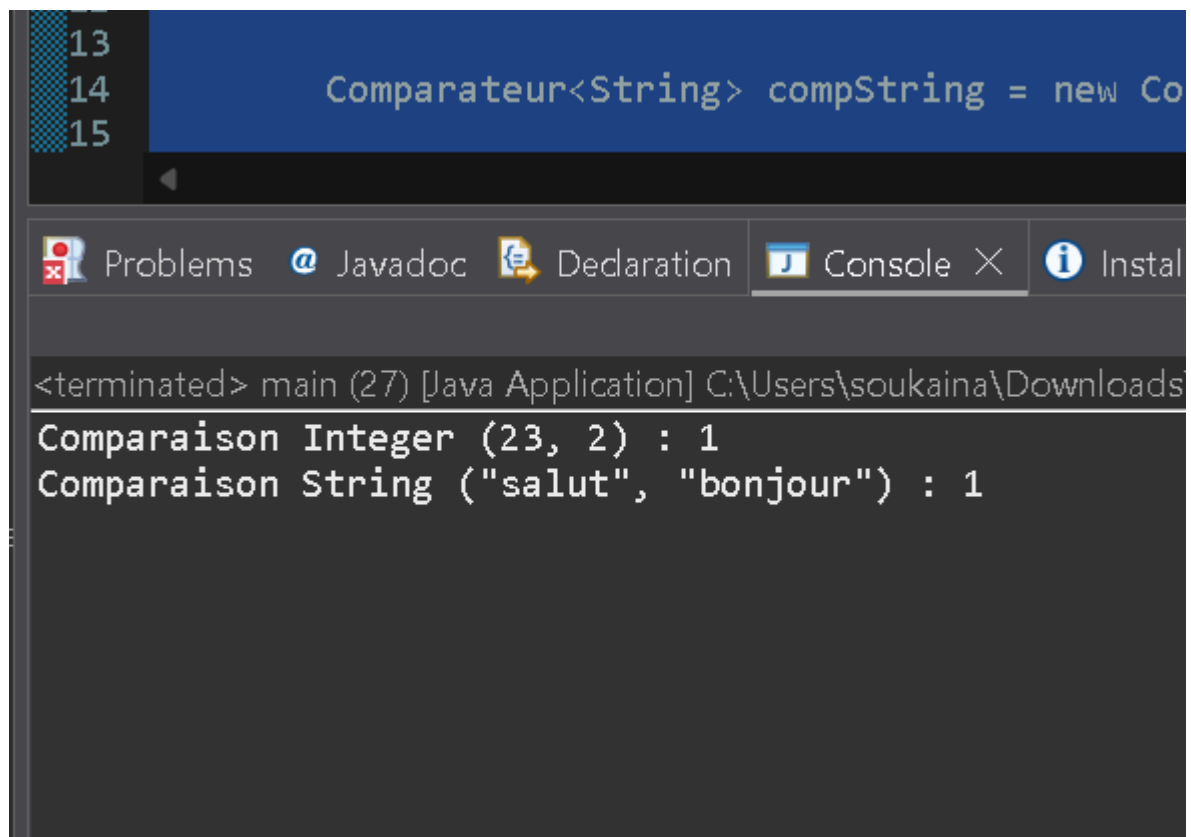
```

package exercice4;
public class main {
    public static void main(String[] args) {

        Compareur<Integer> compInt = new CompareurInteger();
        System.out.println("Comparaison Integer (23, 2) : "
            + compInt.comparer(23, 2));

        Compareur<String> compString = new CompareurString();
        System.out.println("Comparaison String (\\"salut\\", \\"bonjour\\") : "
            + compString.comparer("salut", "hi"));
    }
}

```



```

13
14      Compareur<String> compString = new Co
15
Problems  Javadoc  Declaration  Console  Instal
<terminated> main (27) [Java Application] C:\Users\soukaina\Downloads
Comparaison Integer (23, 2) : 1
Comparaison String ("salut", "bonjour") : 1

```

Exercise 6 :

```

package exercice6;
public class Tableau {

```

```

public static <T> void afficherTableau(T[] tableau) {
    for (T element : tableau) {
        System.out.println(element);
    }
}

public static <T> T getPremier(T[] tableau) {
    if (tableau == null || tableau.length == 0) {
        return null;
    }
    return tableau[0];
}
}

```

```

package exercice6;
public class main {
    public static void main(String[] args) {

        String[] noms = {"omar", "adnane", "malak"};
        System.out.println("Affichage tableau String :");
        Tableau.afficherTableau(noms);
        System.out.println("Premier élément : "
            + Tableau.getPremier(noms));

        Integer[] nombres = {10, 20, 30};
        System.out.println("\nAffichage tableau Integer :");
        Tableau.afficherTableau(nombres);
        System.out.println("Premier élément : "
            + Tableau.getPremier(nombres));
    }
}

```

```
10      System.out.println("Affichage tableau Integer :");
11      Tableau.afficherTableau(noms);
12
13      System.out.println("Premier élément : " +
14                          + Tableau.getPremier(noms));
15  }
```

Problems Javadoc Declaration Console

<terminated> main (32) [Java Application] C:\Users\soukaina\...

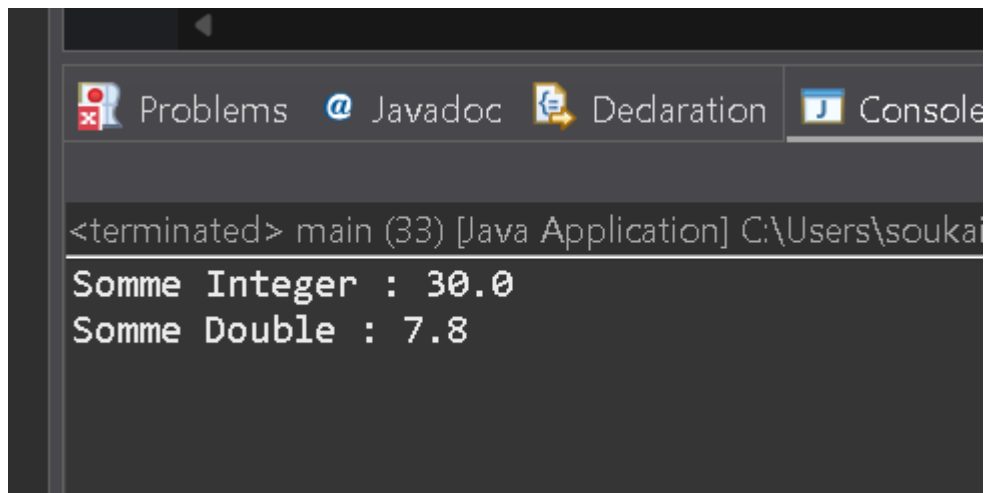
Affichage tableau String :
omar
adnane
malak
Premier élément : omar

Affichage tableau Integer :
10
20
30
Premier élément : 10

Exercice 7 :

```
package exercice7;
public class calcul{
    public static <T extends Number> double somme(T a, T b) {
        return a.doubleValue() + b.doubleValue();
    }
}
```

```
package exercice7;
public class main {
    public static void main(String[] args) {
        System.out.println("Somme Integer : "
            + calcul.somme(10, 20));
        System.out.println("Somme Double : "
            + calcul.somme(5.5, 2.3));
    }
}
```



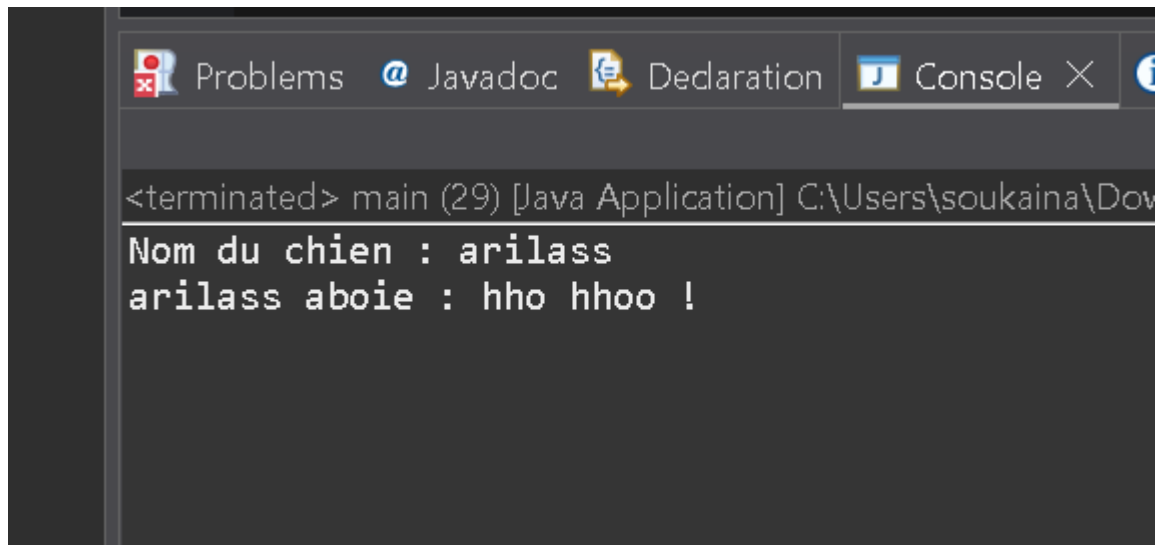
Exercice 8 :

```
package exercice8;
public class Animal<T> {
    T nom;
    public Animal() {
    }
    public Animal(T nom) {
        this.nom = nom;
    }
    public T getNom() {
        return nom;
    }
    public void setNom(T nom) {
        this.nom = nom;
    }
}
```

```
package exercice8;
public class Chien extends Animal<String> {
    public Chien() {
        super();
    }
    public Chien(String nom) {
        super(nom);
    }
    public void aboyer() {
        System.out.println(getNom() + " aboie : hho hhoo !");
    }
}
```

```
package exercice8;
public class main {
    public static void main(String[] args) {
        Chien chien = new Chien("arilass");
        System.out.println("Nom du chien : " + chien.getNom());
        chien.aboyer();
    }
}
```

```
}
```



```
<terminated> main (29) [Java Application] C:\Users\soukaina\Do  
Nom du chien : arilass  
arilass aboie : hho hhoo !
```

Exercice 9 :

```
package exercice9;  
public class Vehicule<T> {  
    T vitesse;  
    public Vehicule() {  
    }  
    public Vehicule(T vitesse) {  
        this.vitesse = vitesse;  
    }  
    public T getVitesse() {  
        return vitesse;  
    }  
    public void setVitesse(T vitesse) {  
        this.vitesse = vitesse;  
    }  
}
```

```
package exercice9;  
public class Voiture<T> extends Vehicule<T> {  
    public Voiture() {  
        super();  
    }  
    public Voiture(T vitesse) {  
        super(vitesse);  
    }  
    public void afficherVitesse() {  
        System.out.println("Vitesse : " + getVitesse());  
    }  
}
```

```
package exercice9;
```

```

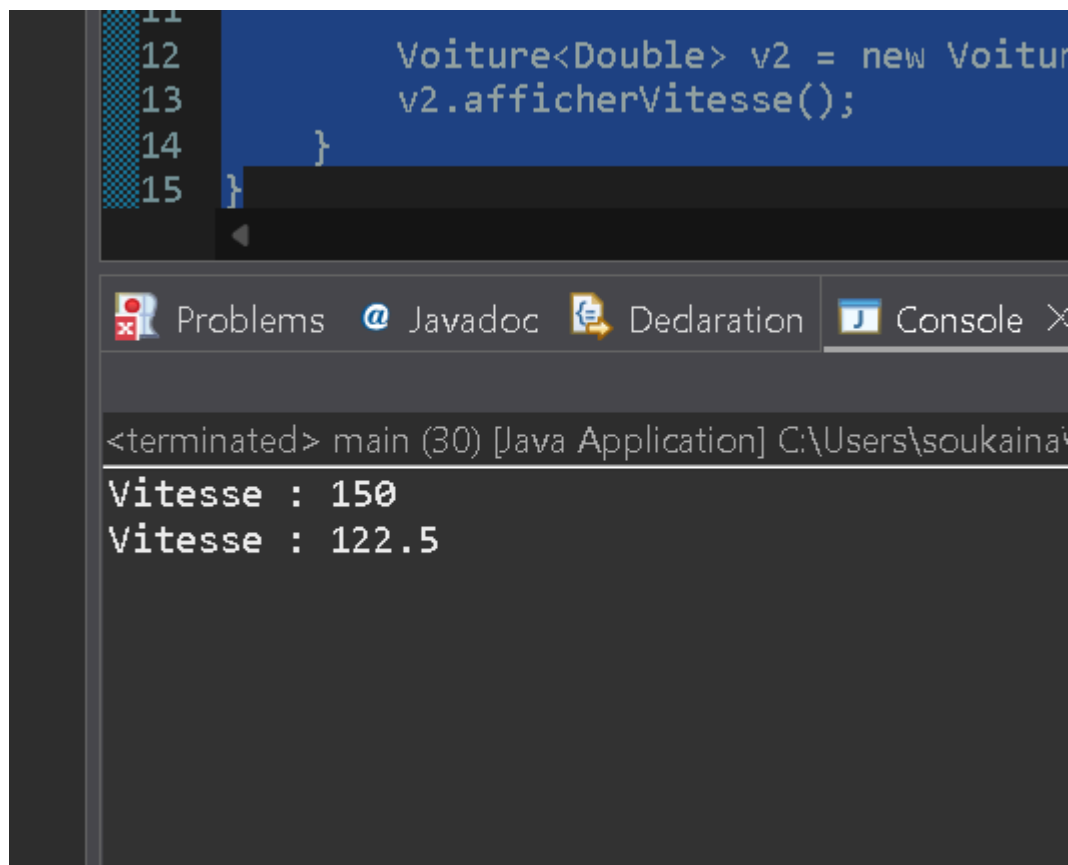
public class main {
    public static void main(String[] args) {

        Voiture<Integer> v1 = new Voiture<>(150);
        v1.afficherVitesse();

        Voiture<Double> v2 = new Voiture<>(122.5);
        v2.afficherVitesse();

    }
}

```



The screenshot shows an IDE with a code editor and a console window. The code editor displays the following Java code:

```

11
12         Voiture<Double> v2 = new Voiture<>(122.5);
13         v2.afficherVitesse();
14     }
15 }

```

The console window shows the output of the program:

```

<terminated> main (30) [Java Application] C:\Users\soukaina\
Vitesse : 150
Vitesse : 122.5

```

Exercice 10 :

```

package exercice10;
public class Repository<T> {
    public void save(T obj) {
        System.out.println("Objet sauvegardé : " + obj);
    }
}

```

```

package exercice10;
public class user {
    private String nom;
    public user(String nom) {

```

```

    this.nom = nom;
}
public String getNom() {
    return nom;
}
@Override
public String toString() {
    return "User{nom=" + nom + "}";
}
}

```

```

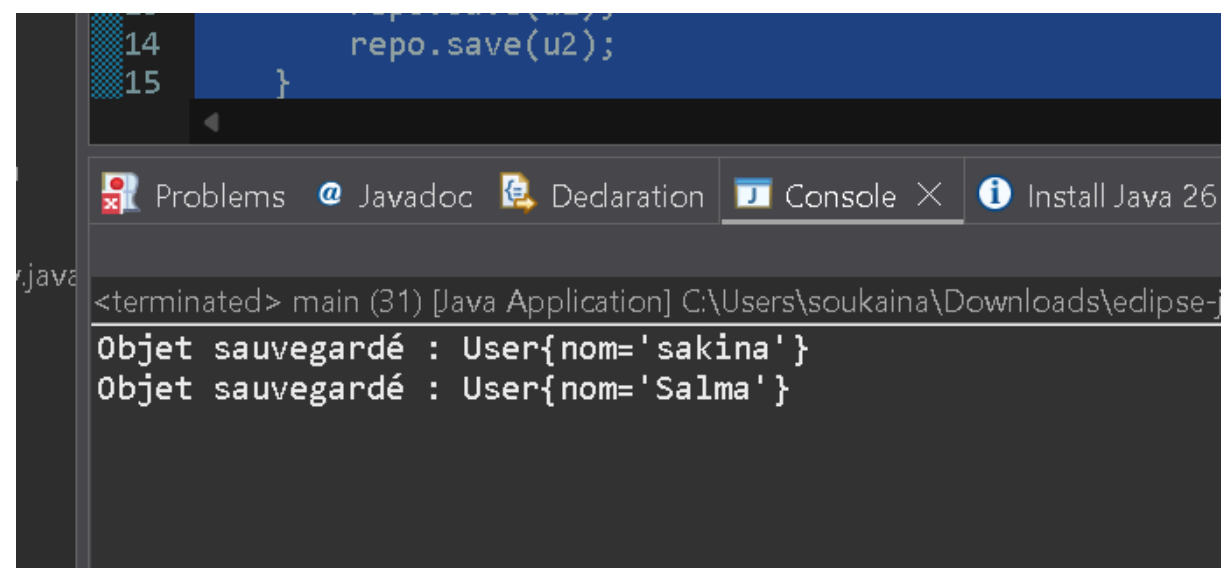
package exercice10;
public class UserRepository extends Repository<user> {
}

```

```

package exercice10;
public class main {
    public static void main(String[] args) {
        UserRepository repo = new UserRepository();
        user u1 = new user("sakina");
        user u2 = new user("Salma");
        repo.save(u1);
        repo.save(u2);
    }
}

```



```

14         repo.save(u2);
15     }

```

Problems Javadoc Declaration Console X Install Java 26

<terminated> main (31) [Java Application] C:\Users\soukaina\Downloads\edipse-j

Objet sauvegardé : User{nom='sakina'}

Objet sauvegardé : User{nom='Salma'}

Exercice 11 :

```

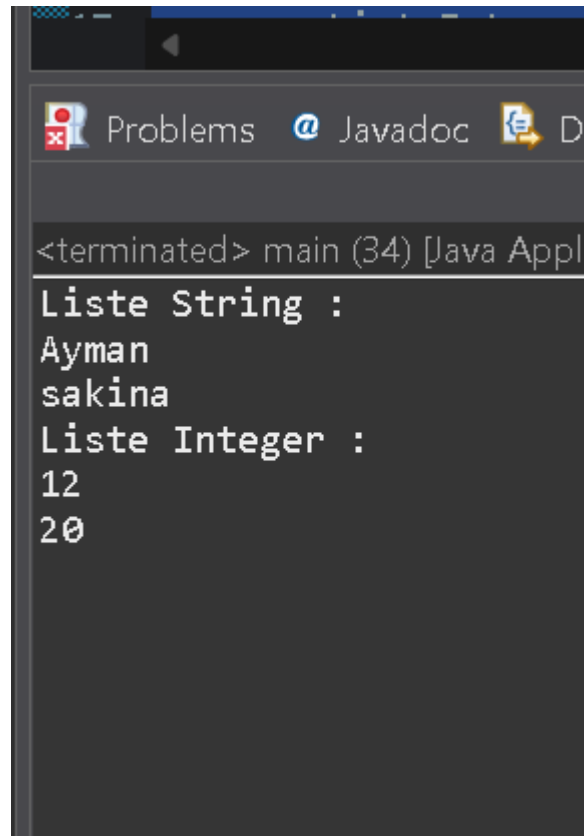
package exercice11;
import java.util.List;
public class ListeUtils {
    public static void afficherListe(List<?> liste) {
        for (Object element : liste) {
            System.out.println(element);
        }
    }
}

```



```
}  
}  
}
```

```
package exercice11;  
import java.util.ArrayList;  
import java.util.List;  
public class main {  
    public static void main(String[] args) {  
  
        List<String> noms = new ArrayList<>();  
        noms.add("Ayman");  
        noms.add("sakina");  
  
        List<Integer> nombres = new ArrayList<>();  
        nombres.add(12);  
        nombres.add(20);  
        System.out.println("Liste String :");  
        ListeUtils.afficherListe(noms);  
        System.out.println("Liste Integer :");  
        ListeUtils.afficherListe(nombres);  
    }  
}
```



```
<terminated> main (34) [Java Appl  
Liste String :  
Ayman  
sakina  
Liste Integer :  
12  
20
```

Exercice 12 :

```
package exercice12;
import java.util.List;
public class Liste {
    public static void afficherNombres(List<? extends Number> liste) {
        for (Number n : liste) {
            System.out.println(n);
        }
    }
}
```

```
package exercice12;
import java.util.ArrayList;
import java.util.List;
public class main {
    public static void main(String[] args) {

        List<Integer> entiers = new ArrayList<>();
        entiers.add(23);
        entiers.add(2);

        List<Double> reels = new ArrayList<>();
        reels.add(5.5);
        reels.add(2.3);
        System.out.println("Liste Integer :");
        Liste.afficherNombres(entiers);
        System.out.println("Liste Double :");
        Liste.afficherNombres(reels);
    }
}
```

15
16

Problems Javadoc Declaration Console X Install Java 26 Support

<terminated> main (35) [Java Application] C:\Users\soukaina\Downloads\edipse-jee-2023

Liste Integer :

23

2

Liste Double :

5.5

2.3

[re]